

```

    lseek(sfs_handle, sb->partition_size * sb->block_size - 1,
SEEK_SET);
    write(sfs_handle, &c, 1); /* To make the file size to
partition size */
}

int main(int argc, char *argv[])
{
    int sfs_handle;

    if (argc != 2)
    {
        fprintf(stderr, "Usage: %s <partition size in 512-byte
blocks>\n", argv[0]);
        return 1;
    }

    sb.partition_size = atoi(argv[1]);
    sb.entry_table_size = sb.partition_size * SFS_ENTRY_RATIO;
    sb.entry_count = sb.entry_table_size * sb.block_size /
sb.entry_size;
    sb.data_block_start = SFS_ENTRY_TABLE_BLOCK_START +
sb.entry_table_size;

    sfs_handle = creat(SIMULA_DEFAULT_FILE, S_IRUSR | S_IWUSR |
S_IRGRP | S_IROTH);
    if (sfs_handle == -1)
    {
        perror("No permissions to format");
        return 2;
    }

    write_super_block(sfs_handle, &sb);
    clear_file_entries(sfs_handle, &sb);
    mark_data_blocks(sfs_handle, &sb);
    close(sfs_handle);
    return 0;
}

```

Note the heuristic of 10 per cent of total space to be used for the file entries, defined by `SFS_ENTRY_RATIO`. Apart from

that, this format application takes the partition size as the input and accordingly creates the default file `.sfsf`, with:

- Its first block as the super block, using `write_super_block()`
  - The next 10 per cent of total blocks as the file entry's zeroed out table, using `clear_file_entries()`
  - And the remaining as the data blocks, using `mark_data_blocks()`. This basically writes a zero at the end, to actually extend the underlying file `.sfsf` to the partition size
- As any other `format/mkfs` application, `format_sfs.c` is compiled using gcc, and executed on the shell as follows:

```

$ gcc format_sfs.c -o format_sfs
$ ./format_sfs 1024 # Partition size in blocks of 512-bytes
$ ls -al # List the .sfsf created with a size of 512 KiBytes

```

Figure 1 shows the `.sfsf` pictorially for the partition size of 1024 blocks of 512 bytes each.

## Summing up

With the above design of a simula file system (`sfs_ds.h`), along with the implementation for its format command (`format_sfs.c`), Shweta has created the empty file system over the simulated partition `.sfsf`. Now, as listed earlier, the final step in simulating the user space file system is to create the interface/shell to type commands and operate on the empty file system just created on `.sfsf`. Shweta will be soon coding that as well. 

### By: Anil Kumar Pugalía

The author is a freelance trainer in Linux internals, Linux device drivers, embedded Linux and related topics. Prior to this, he was at Intel and Nvidia. A gold medalist from the Indian Institute of Science, Linux and knowledge sharing are two of his many passions. Creating and playing around with open source hardware is one of his hobbies. Learn more about him at <http://profession.sarika-pugs.com>, including information about his open source hardware freak-outs and his various Linux related training sessions and workshops. He can be reached at [email@sarika-pugs.com](mailto:email@sarika-pugs.com).

TechnoMail - Enterprise Email Server  
Anti SPAM, Anti Virus, Email Content Filtering

Firewall, Internet Access Control  
Content Filtering, Site Blocking

Bandwidth Management System

Managed Email Hosting Solutions

# TechnoInfotech®

1, Vikas Permisses, 11 Bank Street, Fort, Mumbai, India - 400 001. Mobile: 09167399917. [info@technoinfotech.com](mailto:info@technoinfotech.com)